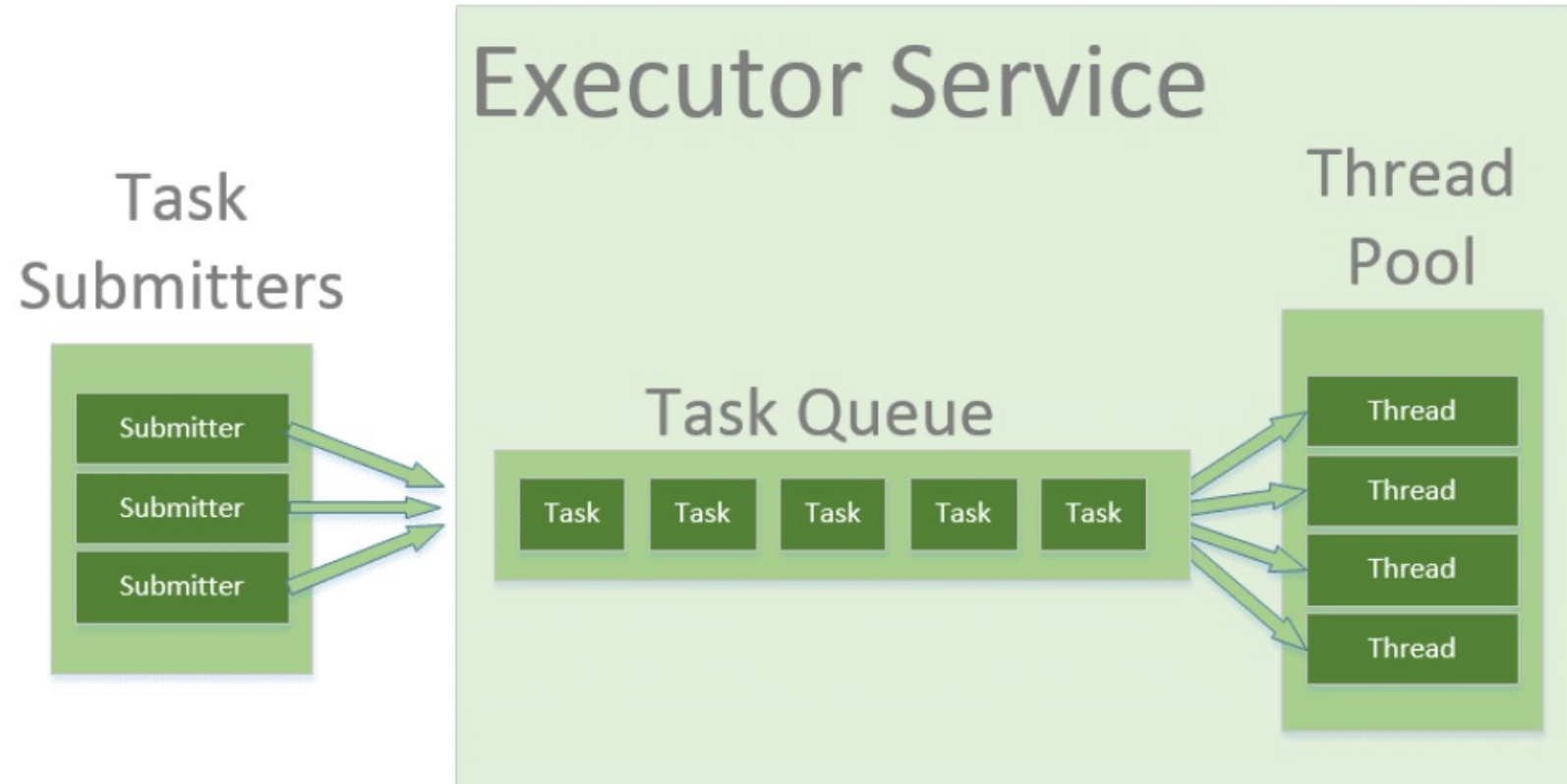




MultiThreading

Java Executor Framework

Java Executor Framework



Java Executor Framework

Java provides its own multithreading framework called the **Java Executor Framework** which is introduced in JDK5 in `java.util.concurrent` package.

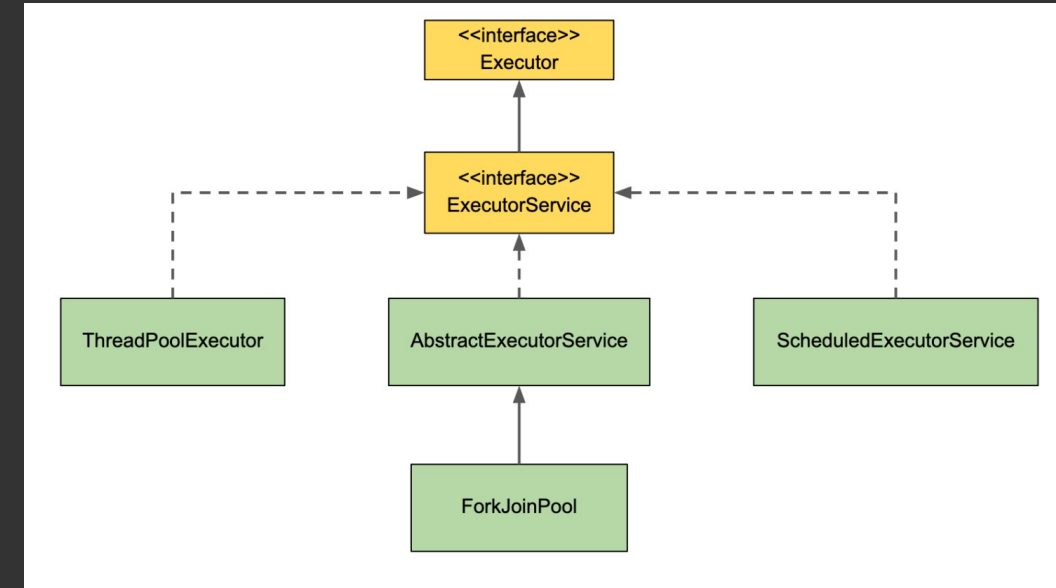
The Executor Framework in Java provides a high-level abstraction for managing concurrent tasks in a Java application. It simplifies multi-threading by handling tasks such as thread creation, management and task scheduling. One of its key features is the thread pool, which reuses existing threads instead of creating new ones, improving efficiency and resource management.

Executor Interface

Executor:- The Executor Interface is at the core of the framework and it provides the `execute method(Runnable)` which takes the tasks and then submits them to the thread pool for execution.

execute(Runnable object): This function schedules the execution of a given command to occur sometime in the future. The command might run in a new thread, a pooled thread, or the calling thread, depending on how the executor is implemented.

```
executor.execute(threadObject);
```

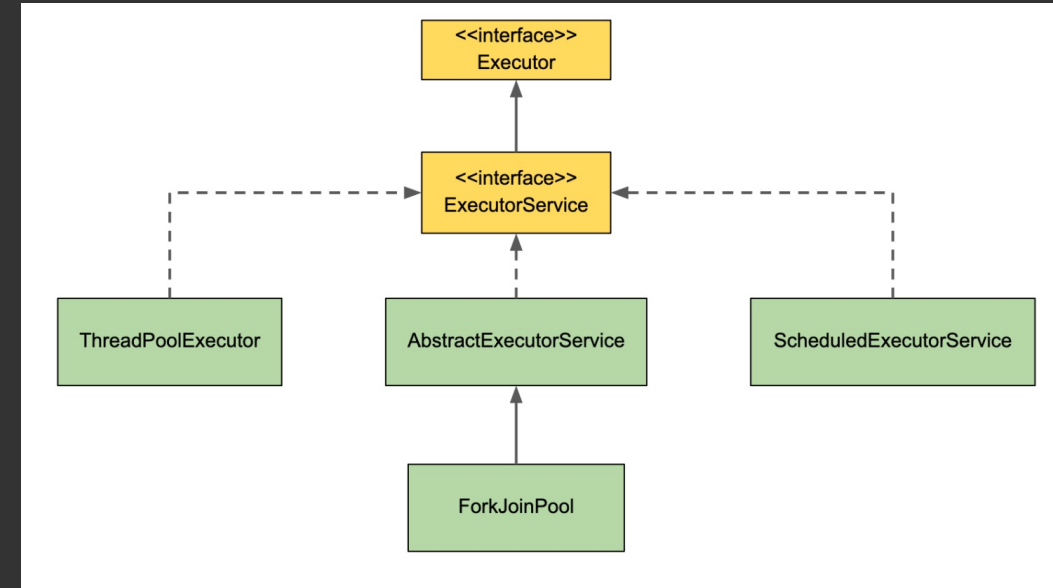


ExecutorService Interface

ExecutorService : The ExecutorService interface is an extension of the Executor interface. It offers a more versatile method called submit, which is similar to execute.

In addition to accepting Runnable objects, submit can also handle Callable objects, allowing tasks to return values.

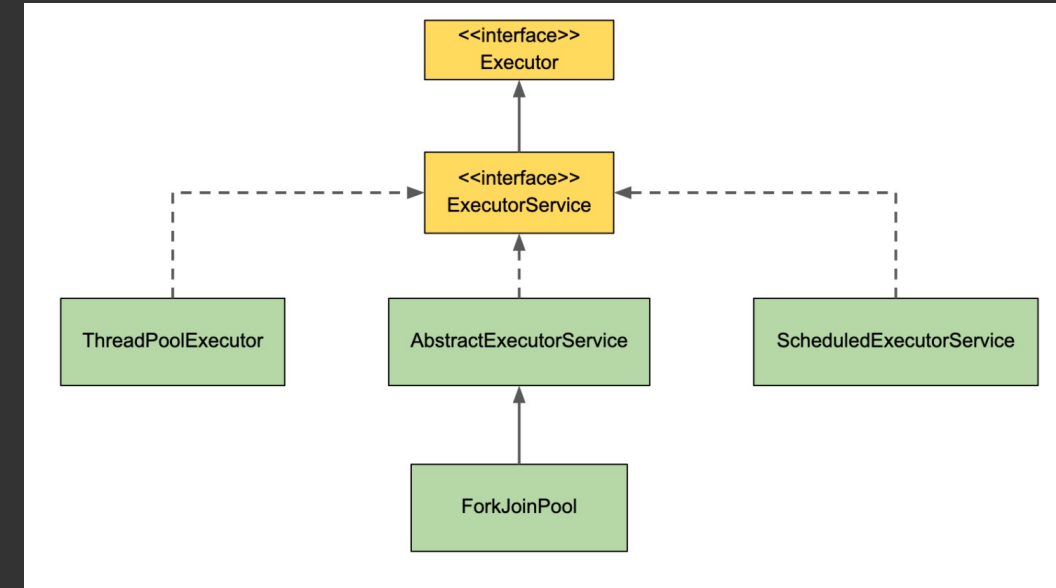
```
Future<String> result =  
executorService.submit(callableTask);
```



ThreadPoolExecutor Class

ThreadPoolExecutor:- This is one of the most commonly used implementations of the `ExecutorService` interface. It manages a pool of worker threads and provides fine-grained control over thread management, allowing you to specify parameters like the core pool size, maximum pool size, and various queueing strategies.

Java thread pool manages the pool of worker threads. It contains a queue that keeps tasks waiting to get executed.



Creating Thread Pool

Create via Executors class:

- `ExecutorService executor = Executors.newFixedThreadPool(5);`
- `ScheduledExecutorService scheduler = Executors.newScheduledThreadPool(2);`
- `ExecutorService executor = Executors.newSingleThreadExecutor();`

Create via **ThreadPoolExecutor** class:

```
ThreadPoolExecutor executorPool = new ThreadPoolExecutor(  
    2, // Core threads  
    4, // Max threads  
    10, // Keep-alive time  
    TimeUnit.SECONDS,  
    new ArrayBlockingQueue<Runnable>(2), // Queue size  
    threadFactory, // Chef hiring process  
    rejectionHandler // Handling extra customers  
);
```

ScheduledExecutorService Class

ScheduledExecutorService: This extends the Executor framework to support scheduling tasks to run at specific times or with fixed time intervals. It's particularly useful for implementing tasks that need to be executed periodically, such as cleanup tasks or periodic data updates.

